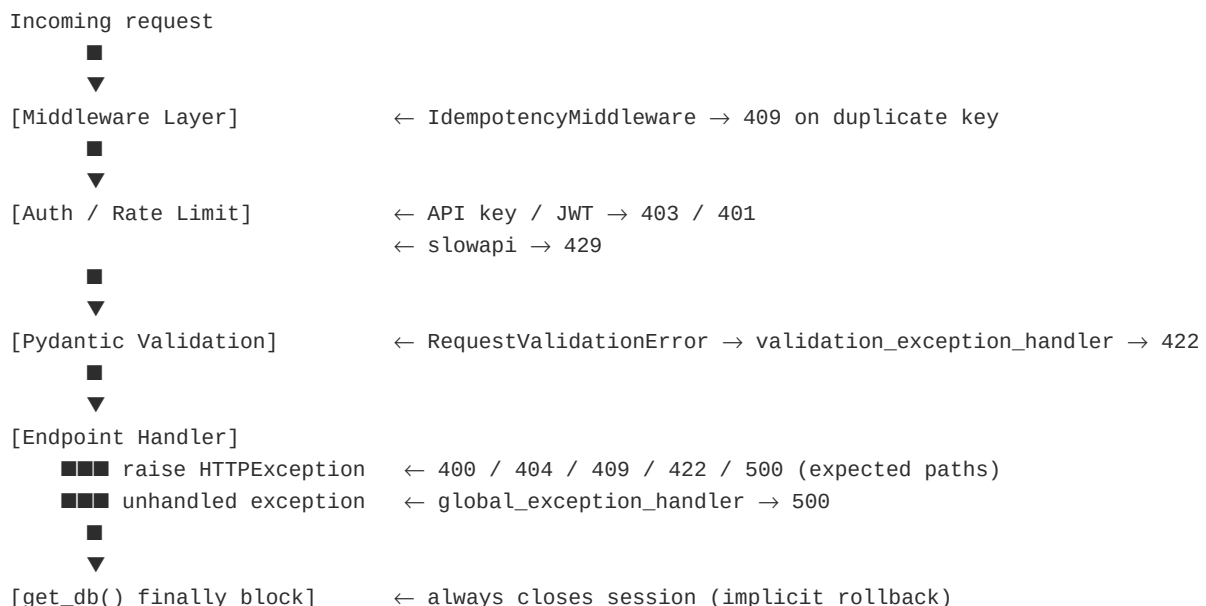


Q8: Global Error-Handling Strategy & Logging

***Question:** What is the global error-handling strategy? Identify any custom exception handlers or middleware used to format error responses. How is logging configured? Where do the logs go, and what level of detail is captured during a standard request lifecycle and during a hardware ingestion failure?*

Part 1: Error-Handling Architecture

The system uses a three-layer defense: **middleware intercepts** before the handler, **HTTPException raises** inside handlers for expected failure paths, and **global exception handlers** in `main.py` catch anything that escapes.



Layer 1: Global Exception Handlers (`app/main.py`)

Two handlers are registered directly on the `FastAPI` app instance.

Handler A: Pydantic Validation Errors

```
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    return JSONResponse(
        status_code=422,
        content={
            "detail": exc.errors(),    # list of field-level errors with loc, msg, type
            "body": str(exc.body) if hasattr(exc, "body") else None,
```

```

        "message": "Validation Error"
    }
)

```

Fires whenever FastAPI fails to deserialize or validate the request body against a Pydantic schema. Returns a structured 422 response with a `detail` array — each entry names the exact field (`loc`), the human-readable error message (`msg`), and the error type (`type`). The raw rejected body is also included for debugging.

This handler does **not** log — validation failures are expected inputs from misconfigured clients.

Example response (wrong type on `passenger_count`):

```

{
  "detail": [
    {
      "loc": ["body", "passenger_count"],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ],
  "body": "{\"trip_id\": 1, \"passenger_count\": \"forty-six\"}",
  "message": "Validation Error"
}

```

Handler B: Catch-All Unhandled Exceptions

```

@app.exception_handler(Exception)
async def global_exception_handler(request, exc):
    logger.error(f"Unhandled exception: {exc}", exc_info=True)
    return JSONResponse(
        status_code=500,
        content={
            "message": "Internal Server Error",
            "detail": "An unexpected error occurred. Please try again later."
        }
    )

```

Catches any exception not handled elsewhere. It always:

1. Logs the exception at `ERROR` level **with** `exc_info=True` — this captures the full Python stack trace in the log output.
2. Returns a generic, non-leaking 500 response — internal details are never exposed to the client.

The database session is **not** rolled back here explicitly; `get_db()` 's `finally` block handles that automatically.

Layer 2: `HTTPException` — Expected Error Paths

All anticipated failure conditions raise `HTTPException` directly inside endpoint handlers or dependency functions. FastAPI's built-in handler serializes these to:

```

{"detail": "<message string>"}

```

No custom formatting — the default FastAPI behavior is used for all `HTTPException` paths.

Status Code Map

Code	Where Raised	Condition
400	Auth, Driver	Inactive user, incorrect old password, wrong driver state
401	<code>deps.py</code>	JWT decode fails (<code>JWError</code>), sub claim missing, user not in DB
403	<code>deps.py</code> , <code>hardware.py</code>	Role mismatch, missing/wrong <code>X-Hardware-API-Key</code>
404	All routers	Entity not found (trip, vehicle, driver, notification)
409	<code>IdempotencyMiddleware</code>	Duplicate Idempotency-Key on POST/PUT/PATCH
422	<code>main.py</code> handler	Pydantic schema validation failure
422	<code>admin.py</code>	Business-logic pre-flight (no active routes for schedule gen)
429	<code>slowapi</code>	Rate limit exceeded per endpoint
500	<code>admin.py</code> , <code>hardware.py</code>	Internal failure that the handler catches and re-raises explicitly

The Hardware API Key Check

```
async def verify_hardware_api_key(x_hardware_api_key: Optional[str] = Header(None)):
    key = x_hardware_api_key or ""
    if not secrets.compare_digest(key, settings.HARDWARE_API_KEY):
        raise HTTPException(
            status_code=status.HTTP_403_FORBIDDEN,
            detail="Invalid or missing hardware API key"
        )
```

This runs as a router-level dependency before any handler body executes. A missing or wrong key returns `403` and no DB session is ever opened.

Layer 3: Middleware Error Paths

SanitizationMiddleware

On malformed JSON (unparseable body), the middleware calls `pass` and lets the raw body through to FastAPI. FastAPI then tries to parse it and produces a `RequestValidationError` , which the

`validation_exception_handler` catches. No silent swallowing of data errors.

IdempotencyMiddleware

```
is_new = await self.redis.set(redis_key, "processing", ex=86400, nx=True)
if not is_new:
    logger.warning(f"Blocking duplicate request: {idem_key}")
    return JSONResponse(status_code=409, content={"detail": "Duplicate request."})

try:
    response = await call_next(request)
    if response.status_code >= 500:
        await self.redis.delete(redis_key) # allow safe retry on server error
    return response
except Exception:
    await self.redis.delete(redis_key)      # allow retry if handler crashes
    raise
```

Two important behaviors:

- If the upstream handler returns `5xx`, the idempotency key is **deleted** so the client can safely retry.
- If the handler raises an unhandled exception, the key is also deleted before re-raising — the global exception handler then catches the re-raised exception and logs it.

Layer 4: Database Session Cleanup

```
async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
        finally:
            await session.close() # always runs – returns connection to pool
```

SQLAlchemy's `async with AsyncSessionLocal()` context manager automatically rolls back any uncommitted transaction when the session is closed without a prior `commit()`. This means:

- If a handler raises `HTTPException` mid-way → no partial writes survive
- If the global exception handler catches an unhandled exception → no partial writes survive
- The `finally` block is guaranteed to run regardless of which exception path exits the handler

Part 2: Logging Configuration

How Loggers Are Declared

Every module that emits logs declares a named logger at module level:

```
# In app/main.py, app/api/v1/hardware.py, app/services/scheduler.py, etc.
import logging
logger = logging.getLogger(__name__)
```

`__name__` resolves to the Python module path (e.g., `app.api.v1.hardware` , `app.services.rotation_service`). This means log output includes the origin module by default, making it trivial to filter logs from a specific subsystem.

Where Logs Go

There is **no explicit** `logging.basicConfig()` , `dictConfig()` , `FileHandler` , **or log rotation** configured in the application code. The logging configuration is entirely delegated to **uvicorn**, which sets up two loggers when the server starts:

Logger name	Content	Default destination
<code>uvicorn.access</code>	One line per HTTP request: <code>method path status_code response_time</code>	<code>stdout</code>
<code>uvicorn.error</code>	All application-level log output (WARNING and above by default)	<code>stderr</code>

All `logger.info()` , `logger.warning()` , `logger.error()` , `logger.critical()` calls in the application propagate up to uvicorn's root handler and appear on **stdout/stderr of the uvicorn process**. In production this is typically captured by the host OS (systemd journal, Docker log driver, etc.).

`engine = create_async_engine(..., echo=False)` — SQLAlchemy's SQL echo is explicitly disabled, so raw SQL queries are never emitted to the log.

Log Levels in Use

The codebase uses four of the five standard levels:

Level	Where Used	Example
CRITICAL	<code>hardware.py</code>	Over-speed GPS event (> 80 km/h)
ERROR	<code>main.py</code> , <code>sockets.py</code> , <code>scheduler.py</code> , <code>hardware.py</code>	Unhandled exception (with stack trace), Redis failure, invalid vehicle capacity
WARNING	<code>main.py</code> , <code>middleware</code> , <code>rotation_service.py</code> , <code>sockets.py</code>	Scheduler startup failure, idempotency block, auto-dispatch failure, trip window overflow
INFO	<code>scheduler.py</code> , <code>sockets.py</code> , <code>rotation_service.py</code>	Job start/success, WS connect/disconnect, rotation swap, gate AUDIT lines

`DEBUG` is **not used** anywhere in the application code.

Part 3: Log Output During a Standard Request Lifecycle

For a typical authenticated request (e.g., `GET /api/v1/admin/dashboard/stats`):

```
# uvicorn.access logger (automatic, one line per request)
INFO:      10.0.0.5:51234 - "GET /api/v1/admin/dashboard/stats HTTP/1.1" 200 OK

# No application-level logger output – happy path produces no entries
```

For a `POST /api/v1/auth/login` with wrong password:

```
INFO:      10.0.0.5:51235 - "POST /api/v1/auth/login HTTP/1.1" 401 Unauthorized
# No app-level log – HTTPException is expected, not logged
```

For a rate-limit breach (slowapi):

```
WARNING:   SlowAPI: Rate limit exceeded for 10.0.0.5
INFO:      10.0.0.5:51236 - "POST /api/v1/auth/login HTTP/1.1" 429 Too Many Requests
```

Part 4: Log Output During a Hardware Ingestion Failure

The `POST /api/v1/hardware/camera` endpoint is the richest failure surface. Here is what the log stream looks like for each failure mode:

Failure: Missing API key

```
# No app-level log emitted – HTTPException 403 raised in dependency
INFO:      <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 403 Forbidden
```

Failure: Pydantic validation error (wrong field type)

```
# No app-level log – validation_exception_handler does not log
INFO:      <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 422 Unprocessable Entity
```

Failure: Trip not found

```
# No app-level log – HTTPException 404, expected path
INFO:      <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 404 Not Found
```

Failure: Vehicle has zero or null capacity

```
ERROR:     app.api.v1.hardware: Vehicle 7 has invalid capacity (0). Skipping crowding calculation.
# Request still returns 200 – crowding_score is set to -1.0 as a sentinel
INFO:      <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 200 OK
```

Failure: Auto-dispatch — no DRIVER_3 available

```
WARNING:   app.services.rotation_service: Auto-dispatch failed for trip 42: No available D3 drivers.
# Request still returns 200 – camera reading is still persisted
INFO:      <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 200 OK
```

Failure: Auto-dispatch — no free vehicle available

```
WARNING: app.services.rotation_service: Auto-dispatch failed for trip 42: No available vehicles.
INFO:    <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 200 OK
```

Failure: Over-speed GPS event (not a failure, but a critical alert)

```
CRITICAL: app.api.v1.hardware: Vehicle 3 over-speed: 94.5 km/h [coordinates redacted for PII]
INFO:    <device_ip> - "POST /api/v1/hardware/gps HTTP/1.1" 204 No Content
```

Note the PII comment in the log line — latitude/longitude are deliberately omitted from the log message to avoid storing location data in logs.

Failure: Unhandled exception (e.g., DB connection lost mid-request)

```
ERROR:    app.main: Unhandled exception: asyncpg.exceptions.ConnectionDoesNotExistError: ...
Traceback (most recent call last):
  File ".../app/api/v1/hardware.py", line 276, in ingest_camera
    await db.commit()
  File ".../sqlalchemy/ext/asyncio/session.py", line ...
    ...
asyncpg.exceptions.ConnectionDoesNotExistError: connection was closed in the middle of operation

INFO:    <device_ip> - "POST /api/v1/hardware/camera HTTP/1.1" 500 Internal Server Error
```

The `exc_info=True` flag causes Python's logging module to append the full traceback. This is the only path that produces a multi-line log entry. The database session is closed by `get_db()`'s `finally` block — any uncommitted writes are rolled back automatically.

Summary: Who Handles What

Error Type	Handler	Logged?	DB Rolled Back?
Missing/wrong API key	<code>verify_hardware_api_key</code> dep	No	Session never opened
JWT invalid/expired	<code>get_current_user</code> dep	No	Session never opened
Role mismatch	<code>get_current_user_with_role</code> dep	No	N/A
Pydantic schema failure	<code>validation_exception_handler</code>	No	Session closed cleanly
Business rule violation	<code>raise HTTPException()</code> in handler	No	Session closed cleanly
Rate limit exceeded	<code>slowapi</code>	WARNING (slowapi internal)	Session never opened
Duplicate idempotency key	<code>IdempotencyMiddleware</code>	WARNING	Session never opened
Hardware capacity=0	Inline guard in handler	ERROR	No (request succeeds)

Error Type	Handler	Logged?	DB Rolled Back?
Auto-dispatch unavailable	<code>trigger_auto_dispatch</code> return	WARNING	No (partial commit done)
Unhandled exception	<code>global_exception_handler</code>	ERROR + full traceback	Yes (via finally)